How Artificial Intelligence Models Are Trained: Applying Computer Vision to Underwater Vehicle Detection

Introduction

Artificial intelligence (AI) models are trained by using data to adjust a mathematical system so that it can recognize patterns and make predictions on new information. In computer vision, this process allows a model to identify objects within images or videos. Training an AI model for underwater object detection presents additional challenges because water clarity, lighting, distance, and background conditions can significantly affect an image.

The senior design project addresses these challenges by developing a computer vision system that can detect and help locate a disabled REMUS 600 unmanned underwater vehicle (UUV). The proposed system will use a pretrained You Only Look Once (YOLO) object detection model, which will be fine-tuned using underwater photographs of a physical 3D-printed model and computer-generated images. The goal is to determine whether combining real and synthetic images improves detection performance in previously unseen underwater conditions.

1. Preparing the Training Data

The first step in training an AI model is collecting and preparing a dataset. A model cannot reliably learn to recognize an object unless it is provided with representative examples of that object and information about what it should identify. For an object detection model, this process requires images and corresponding annotations that identify the location of each object.

The proposed project will create two datasets. The first will consist of photographs and video frames of a physical, 3D-printed REMUS 600 model submerged in water. Images will be collected under different conditions including varying camera distances; different lighting levels; and different backgrounds. Water clarity and partial obstruction will also be varied to expose the model to challenging conditions. The images will then be manually labeled with bounding boxes, which are rectangular outlines indicating the object's location in each image.

The second dataset will be generated using Blender, a 3D modeling and rendering program, and BlenderProc, a tool for automating scene generation and image rendering. A digital REMUS model will be placed in simulated underwater environments, where scripts can vary the camera angle, object orientation, lighting, visibility, and surrounding objects. Because the object's position is known in each rendered scene, its bounding-box annotations can be generated automatically.

The proposal targets approximately 500–1,000 labeled physical images and 3,000–5,000 synthetic images. Synthetic data can increase dataset size and provide precise labels without requiring every image to be photographed and annotated manually. However, computer-generated images may not perfectly reproduce real underwater effects. For example, actual water can introduce complex light scattering, color changes, and visual distortions that are difficult to simulate accurately. Therefore, synthetic data should supplement rather than automatically replace physical underwater photographs.

The dataset must also include appropriate negative examples, which are images without the REMUS vehicle. These examples help the model learn when it should not report a detection, reducing false positives in scenes containing rocks, debris, or other objects.

2. Training the Model Through Transfer Learning

Rather than building an AI model entirely from scratch, the project proposes fine-tuning an existing pretrained YOLO model. This approach is an example of *transfer learning*, in which a model's previously learned parameters provide a starting point for a new task. A pretrained vision model has already learned useful visual patterns, such as edges, shapes, textures, and object features. Fine-tuning adapts those learned representations to the specific task of detecting the REMUS 600. This can reduce the amount of task-specific data and training time required compared with training an equivalent model from random initialization.

The proposed software tools include Python, PyTorch, Ultralytics YOLO, and OpenCV. YOLO is designed for object detection: given an image, it predicts which objects are present and where they appear. For this project, the model will be configured to recognize the REMUS as the target class.

Training requires the model to process labeled images and compare its predictions with the correct annotations. During each training iteration, the model predicts bounding boxes and associated class probabilities. A loss function measures discrepancies between the predictions and the expected results. An optimization algorithm then updates the model's parameters to reduce this loss. This process repeats over many batches of images. One complete pass through the training dataset is called an *epoch*.

As training progresses, the model adjusts its internal parameters to improve its ability to distinguish the REMUS from the surrounding environment. Data augmentation, such as carefully chosen changes in image brightness, scale, or orientation, can also help expose the model to additional visual variation. However, augmentation should reflect plausible underwater conditions; unrealistic transformations can make training less representative of the intended application.

Training does not guarantee that the model will perform well on every new image. A model can overfit if it learns details specific to the training examples rather than general features of the REMUS. For instance, it might associate the vehicle with a particular background or lighting arrangement. To reduce this risk, the dataset must be divided into separate training, validation, and testing sets, each serving a distinct purpose.

3. Validation and Evaluation

The training set is used to update the model's parameters. The validation set is used during development to compare configurations and guide choices such as training duration and hyperparameters. The test set is reserved for the final evaluation of the selected model. Keeping these sets separate provides a more reliable estimate of performance on unfamiliar images.

For this project, it is particularly important to split physical images by recording session rather than randomly assigning individual video frames. Consecutive frames often contain nearly identical views of the same object. If similar frames appear in both training and testing sets, the evaluation may overestimate how well the model generalizes to new conditions. Grouping images by recording session helps reduce this data leakage.

The proposal identifies four model configurations for comparison:

- **Model A:** Trained using physical underwater photographs only.
- **Model B:** Trained using synthetic Blender images only.
- **Model C:** Trained using a combination of physical and synthetic images.
- **Model D:** Initially trained on synthetic images and then fine-tuned on physical images.

All four configurations will be evaluated on the same unseen physical underwater test dataset. This controlled comparison will help determine whether synthetic data improves detection performance and whether sequential training provides an advantage over combining both datasets from the beginning.

Evaluation should consider multiple metrics. *Precision* measures the proportion of predicted detections that are correct, while *recall* measures the proportion of actual target objects that the model successfully detects. A model with high precision but low recall may miss the vehicle frequently; a model with high recall but low precision may produce too many false alarms. Mean average precision (mAP) provides a broader measure of object detection performance by combining precision-recall behavior with bounding-box overlap criteria. The evaluation should also examine missed detections and false positives under different visibility, lighting, and distance conditions.

Ultimately, the best model should be selected based on measured performance on unseen physical images, not simply on training accuracy or the number of images used.

Conclusion

Training an AI model involves more than feeding images into a program. It requires collecting representative data, creating accurate annotations, selecting an appropriate model, optimizing its parameters, and testing its performance on data it has never encountered. For the REMUS 600 recovery project, combining physical underwater photographs with synthetic images offers a practical way to investigate whether a larger and more varied dataset improves object detection.

By comparing four training configurations under consistent testing conditions, the project can determine which approach provides the most reliable detections in realistic underwater imagery. The final system will then combine those detections with simulated sensor information to estimate the disabled vehicle's location within a 3D environment. Although successful training is an important milestone, reliable

recovery also depends on accurate localization and appropriate integration with the simulation. The results of this project will demonstrate both the potential benefits and the limitations of using computer vision to support underwater vehicle recovery.